https://hjfy.top/arxiv/2601.05505v1

FlashMem：通过计算复用提炼内在潜在记忆

Yubo Hou[1], Zhisheng Chen[2,3], Tao Wan[4], Zengchang Qin[1]*
[1]School of ASEE, Beihang University, Beijing, China
[2]Institute of Computing Technology, Chinese Academy of Sciences, Beijing, China
[3]University of Chinese Academy of Sciences, Beijing, China
[4]School of Biological Science and Medical Engineering, Beijing Advanced Innovation Center for Biomedical Engineering, Beihang University, Beijing, 100191, China
{houyubo, zcqin}@buaa.edu.cn

摘要

大型语言模型的状态无感知架构本质上缺乏保持动态上下文的机制，迫使智能体冗余地重新处理历史信息以维持长时程自主性。尽管潜在记忆提供了解决方案，但现有方法受限于架构分离，依赖辅助编码器，导致记忆与推理主干解耦。我们提出 **FlashMem**，一种通过计算复用从瞬态推理状态中提炼内在记忆的框架。利用内部表示唯一编码输入轨迹的特性，FlashMem 将最后一个隐状态识别为交互历史的充分统计量。这使得一个 **共享-KV 整合器**能够直接关注主干的冻结缓存，合成记忆，从而消除冗余的再参数化。此外，一个无参数的 **认知监控器**利用注意力熵自适应地仅在检测到高认识不确定性时触发整合。实验表明，FlashMem 在性能上可媲美重型基准，同时将推理延迟降低 **5 倍**，有效弥合了效率与持续认知之间的差距。

1 引言

在长时间跨度上维持连贯的认知状态是实现通用自主性的先决条件。尽管大模型(LLMs)作为强大的推理引擎表现出色，但其架构本质上是无状态的：它们基于静态参数将输入映射到输出，且在交互之间不保留持久的内部状态。这种设计造成了模型固定的参数化知识与长时程任务所需的动态过程上下文之间的二元对立。因此，在不断演进的工作流中，智能体在每次推理步骤后被迫丢弃丰富的中间推理表示 (Hu et al., 2025)。这种无状态性导致历史信息被重复处理，为需要持续积累经验并不断适应的智能体带来了显著瓶颈。

为了消除瞬时推理与持续认知之间的差距，记忆机制已成为研究的焦点。当前的方法通常分为参数化、token 级和潜在形式 (Hu et al., 2025)。参数化更新在实时适应中计算开销过大，而 token 级检索（例如，Mem0 (Chhikara et al., 2025)）往往面临信息密度低的问题，导致原始文本占据了有限的上下文窗口，而非合成的理解。针对这些局限性，生成式潜在记忆作为一种有前景的范式应运而生 (Zhang et al., 2025; Xu et al., 2025a)。通过将广泛上下文压缩为稠密的连续向量，这些方法作为紧凑的认知缓冲区，能够在不耗尽注意力预算的情况下保留关键信息。

然而，由于存在架构分离和计算冗余，当前最先进的潜在记忆范式中仍存在结构性低效问题。主流方法通常通过参数发散的辅助模块——无论是独立的编码器 (Wu et al., 2025; Yu et al., 2025) 或专用适配器 (Zhang et al., 2025)——来合成记忆，这些模块与主推理骨干分离。这种分叉结构阻止了对骨干缓存的键值 (KV) 状态的直接复用，迫使系统维护相互隔离的记忆缓冲区，并对上下文重新编码执行重复的前向传播。此外，依赖迭代最优化 (Li et al., 2025) 或全规模自回归解码来生成记忆不可避免地打断了思维的连续性，引入了显著

*Corresponding author.

的推理延迟。我们认为，一种高效的记忆机制应具有内在性——直接从智能体现有的计算流程中获取记忆，以绕过冗余的重参数化过程。

实现这一愿景，我们提出 **FlashMem**，一种通过计算复用直接从智能体的瞬时推理状态中提炼潜在记忆的框架。利用大语言模型内部表示唯一编码输入轨迹的特性 (Nikolaou et al., 2025)，FlashMem 将最后一个隐状态视为交互历史的充分统计量。这使得我们能够采用一个 **共享-KV 整合器**，通过直接关注主干网络的冻结缓存来合成记忆，从而将推理过程本身视为记忆的决定性来源，无需结构上的分离。此外，为使记忆生成与实时认知需求对齐，我们引入了一个无参数的 **认知监控器**。该机制基于注意力熵与模型不确定性之间的相关系数 (Kuhn et al., 2023; Farquhar et al., 2024)，仅在智能体表现出高内部困惑时触发整合。

我们的贡献总结如下：

- 我们提出 **FlashMem**，一个轻量级框架，旨在通过 计算复用提取内在记忆。与需要大量重新编码的分离式架构不同，FlashMem 直接利用主干网络的瞬态推理状态，为长时域智能体提供一种高效解决方案，且无需架构解耦。
- 我们提出了一种极简的 **共享-KV 整合器**，其理论基础是最后一个隐状态的充分性。通过直接从主干网络的冻结缓存中合成记忆，该设计消除了冗余。实验结果表明，它在保持与重型基准相当性能的同时，将推理延迟降低了 **5**× ，且显存开销可忽略不计。
- 我们引入了一个无参数的**认知监控器**，该监控器利用注意力熵作为认知不确定性实时信号。我们的分析表明，高熵能够有效定位模型的混淆和幻觉现象，使监控器仅在必要时自适应地触发记忆巩固，从而避免不必要的计算开销。

2 相关工作

最近的研究将智能体记忆分为 *token-level*、*parametric* 和 *latent* 三种形式 (Hu et al., 2025)。尽管每种形式都推动了该领域的发展，但在其生成和管理方式上仍存在结构性低效问题。

令牌级记忆与参数化记忆。 token 级别的记忆（例如，RAG (Lewis et al., 2020)，MemGPT (Packer et al., 2023)，Mem0 (Chhikara et al., 2025)）提供了透明性，但存在信息密度低的问题，经常用检索到的上下文耗尽有限的注意力预算。相反，参数化记忆（例如，ROME (Meng et al., 2022)）通过将知识内化到权重中，提供了高效的推理能力，但缺乏实时、回合内适应所需的可塑性 (Hu et al., 2025)。

生成式潜在记忆 为了提升密度，研究已转向主动记忆生成，将上下文编码为连续向量。如 MemGen (Zhang et al., 2025) 和 SoftCoT (Xu et al., 2025a) 等代表性工作采用辅助模块——独立的编码器或不同的 LoRA 编辑器——将上下文压缩为概要 token (Mu et al., 2023) 或潜在状态。多模态扩展如 VisMem (Yu et al., 2025) 和 CoMEM (Wu et al., 2025) 也依赖于分离的编码器。然而，这些架构计算开销较大。例如，MemGen 需要同时维护三个独立的模型实例（触发器、编织器和推理器），而 SoftCoT 则依赖自回归解码或迭代最优化来合成提示。因此，它们需要通过二次参数重新编码历史，导致显著冗余和显存占用。

效率与认知控制。 为缓解长上下文推理的高昂开销，研究聚焦于两条最优化路径：状态压缩与选择性激活。如 SnapKV (Li et al., 2024)、H2O (Zhang et al., 2023) 和 PyramidKV (Cai et al., 2025) 等启发式方法通过剪枝低注意力活性值的 token 来压缩键值缓存（KV cache）。尽管快速，这些提取式方法相较于生成式合成，更易破坏长程语义依赖关系。除了压缩之外，优

化激活时机同样至关重要。现有系统通常依赖静态提示或复杂的可训练分类器 (Zhang et al., 2025)，忽视了注意力熵与模型不确定性之间的已知关联 (Kuhn et al., 2023; Farquhar et al., 2024; Guerreiro et al., 2023)。FlashMem 通过集成共享键值（Shared-KV）整合器以实现高效的计算复用，并引入无参数的熵监控器以实现自调节激活，从而解决上述局限。

3 方法学

3.1 preliminaries 和问题表述

智能体的构建与效率差距。 我们将基于大语言模型的智能体形式化为一个参数化策略 π_θ ，通过轨迹 $\tau_{<t} = (o_1, a_1, \ldots, o_{t-1}, a_{t-1})$ 进行交互。在步骤 t ，智能体根据历史 $\tau_{<t}$ 、当前观测 o_t 和辅助记忆 $\mathcal{M}$ 条件下采样动作 a_t ：

$$a_t \sim \pi_\theta(\cdot \mid \tau_{<t}, o_t, \mathcal{M}) \tag{1}$$

在本工作中，我们专注于潜在记忆，其表示为连续嵌入 $\mathcal{M} = \{m_1, \ldots, m_K\}(m_i \in \mathbb{R}^d)$。与基于 token 的检索不同，潜在记忆充当一个高密度缓冲区，以极小的 token 开销编码抽象推理模式。

然而，主流范式通常采用分离的架构，通过辅助过程 $\mathcal{G}_\phi$ 生成记忆：

$$\mathcal{M} = \mathcal{G}_\phi(\tau_{<t}) \tag{2}$$

这需要一次独立的编码过程，导致了 计算冗余，即对 $\tau_{<t}$ 重复编码，以及由于维护分离的 KV 缓存而产生的 状态隔离。我们的目标是通过重用主干网络的状态，内在地合成 $\mathcal{M}$。

3.2 FlashMem 概述

为解决上述效率瓶颈，我们提出 **FlashMem**，一种与大语言模型主干网络 π_θ 紧密耦合的内存生成架构。如图 1 所示，与依赖独立编码过程的分离式系统不同，FlashMem 遵循计算复用原则：直接从主干网络的瞬时活性值状态中合成内存，从而消除重复编码以及维护独立键值缓存所带来的开销。该框架包含两个协同工作的组件：

- **认知监控器（第 3.3 节）：** 一种无参数的门控机制，用于决定在何时生成记忆。当检测到高熵（表明模型出现困惑）时，监控器会暂停标准生成过程，以启动记忆巩固。
- **记忆巩固器（第 3.4 节）：** 一种轻量级解码器，旨在高效地确定如何合成记忆。为避免对长历史轨迹的重复处理，其采用共享键值（KV）设计。具体而言，它将主干网络当前的隐状态 h_t 投影作为初始查询，并直接关注主干网络累积的键值缓存。该机制使 FlashMem 能够直接从主干网络的高层表示中提炼出有价值的知识和经验模式，无需冗余的重新编码。

推理工作流。 推理过程中，FlashMem 动态切换标准生成模式与记忆增强模式。合成的记忆 $\mathcal{M}$ 以软注入方式回传至主干网络的输入流中。这些潜在嵌入作为高密度认知线索，不仅揭示相关的历史上下文，还注入抽象推理模式，引导后续复杂的决策过程。

3.3 认知监视器：基于熵的不确定性感知

当前的生成式记忆范式通常依赖于僵化的触发机制：要么是静态注入——例如 Soft-CoT (Xu et al., 2025a) 无差别地在 token 前添加内容——导致简单查询浪费计算资源，要么是学成分类器如 MemGen (Zhang et al., 2025)，引入了架构开销和分布偏移风险。相反，我们提出利用大语言模型内在的不确定性作为无参数的激活信号。在不确定性估计 (Kuhn et al., 2023) 和幻觉检测 (Farquhar et al., 2024) 方面的开创性工作已表明，模型内部概率分布的离散程度与其语义置信度存在强烈相关性。具体而言，当模型感到困惑或容易虚构时，其注意力机制往往表现出较高的熵值，对关键 token 缺乏聚焦 (Guerreiro et al., 2023;

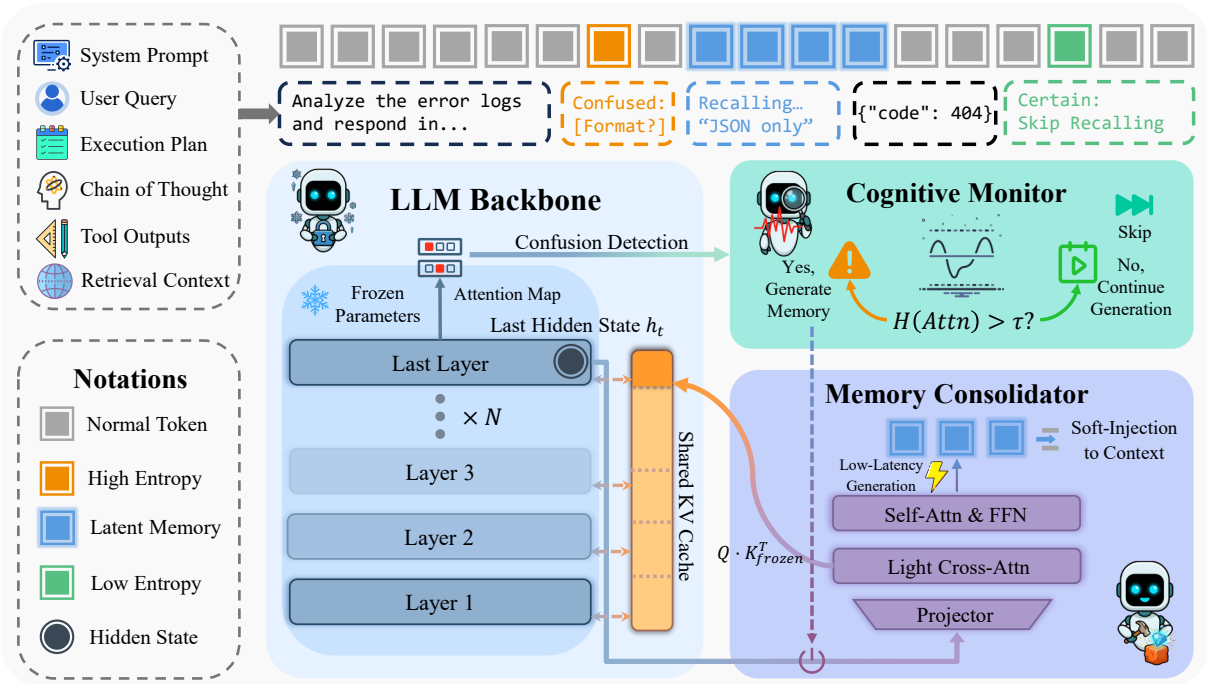

图 1: **FlashMem** 的架构。其特征在于一个无参数的 **认知监控器**（右上角），该模块根据注意力熵不确定性激活记忆生成，以及一个轻量级的 **记忆整合器**（右下角）。该整合器采用 *共享-KV* 设计，通过重用冻结主干网络的缓存，高效地生成潜在记忆，以隐状态 h_t 作为初始种子。

Bibal et al., 2022)。基于这一理论共识，我们采用注意力熵作为实时的元认知指标。

缓解注意力消耗。 然而，由于注意力坍缩现象 (Xiao et al., 2024) 的存在，直接从原始注意力权重计算熵是不可靠的。研究表明，大模型（以及视觉 Transformer (Darcet et al., 2024)）往往会将大量注意力得分分配给初始 token（例如 [BOS]），而不论其语义相关性如何。这种主导性的坍缩点人为地充当了一个低熵锚点，掩盖了推理过程的真实分布情况。为了提取去噪后的认知信号，我们处理最后一层的注意力权重，该层已被证明编码了最丰富的语义信息和任务特定信息 (Tenney et al., 2019)。令 $A_{t,h} \in \mathbb{R}^t$ 表示当前步骤 t 下头 h 在上下文序列上的注意力分布。我们将 $\mathcal{S}_{\text{sink}}$ 定义为对应于注意力坍缩点的索引集合。我们显式地对这些 token 进行掩码，并对每个头重新归一化分布：

$$\tilde{A}_{t,h}[j] = \frac{A_{t,h}[j] \cdot \mathbb{I}(j \notin \mathcal{S}_{\text{sink}})}{\sum_k A_{t,h}[k] \cdot \mathbb{I}(k \notin \mathcal{S}_{\text{sink}})} \tag{3}$$

其中 $\mathbb{I}$ 为指示函数，j, k 表示 token 索引。

熵决策与自适应干预。 我们分别计算每个头的香农熵以捕捉多样的不确定性模式，然后将它们聚合形成最终度量。系统不确定性 $\mathcal{H}_t$ 定义为所有 H 个头上的平均熵：

$$\mathcal{H}_t = \frac{1}{H} \sum_{h=1}^{H} \left(- \sum_{j \notin \mathcal{S}_{\text{sink}}} \tilde{A}_{t,h}[j] \log \tilde{A}_{t,h}[j] \right) \tag{4}$$

只有当 $\mathcal{H}_t > \tau$ 时，记忆巩固才会被激活。阈值 τ 作为灵敏度旋钮，控制着效率与性能之间的权衡。较低的 τ 会促使智能体更积极地寻求记忆辅助（优先考虑准确率），而较高的 τ 则仅在出现严重困惑时才允许干预（优先考虑速度）。这种灵活性使得 FlashMem 能够适应测试时不同的计算预算，相较于固定成本分类器具有显著优势。实际上，我们并未依赖固定的启发式数值，而是采用一种分布感知的校准策略。τ 的最优基准与模型的内在置信度以及任务的开放性相关。我们在附录 C 中讨论了这些熵动态，并详细介绍了我们的统计校准协议。

3.4 记忆巩固器：共享键值生成

激活后，系统旨在高效地合成记忆。为实现这一目标，我们引入了**记忆整合器**，这是一种轻量级解码器架构，严格遵循计算复用原则设计。

通过可注入性进行状态投影。 利用 LLM 表示的单射性 (Nikolaou et al., 2025)，我们采用主干网络的最后隐状态 h_t 作为历史上下文的全面语义摘要。不重新编码输入，而是将主干网络视为预先计算好的编码器。我们通过一个两层 MLP 将 h_t 投影到整合器的工作维度，生成初始潜在嵌入 m_0。这使得整合器能够继承主干网络的理解能力，而无需进行冗余处理。

无投影交叉注意力 合并器由 L 个堆叠的解码器层组成。为最小化参数量和显存占用，我们引入了一种无投影的交叉注意力机制。与需要可训练投影来生成键（Keys）和值（Values）的标准注意力层不同，我们舍弃了 W_K 和 W_V。令 x 表示当前层的输入状态，初始化为 m_0。该模块仅使用 x 计算查询向量，并直接对主干网络累积的键矩阵 $\mathbf{K}$ 和值矩阵 $\mathbf{V}$ 进行注意力计算：

$$\mathrm{Attn}(x, \mathbf{K}, \mathbf{V}) = \mathrm{softmax}\left(\frac{(xW_Q)\mathbf{K}^\top}{\sqrt{d}}\right)\mathbf{V} \tag{5}$$

其中 W_Q 是注意力操作中唯一的可学习投影矩阵。该机制确保了上下文维护的存储开销可以忽略不计，因为生成过程锚定于骨干网络已有的状态池。

权重继承与软注入。 从零开始训练一个轻量级模块，以与一个冻结的大型主干协同工作，面临着显著的语义对齐挑战。随机初始化通常会导致较大的分布偏移，从而引发不稳定的最优化过程。

为缓解这一问题，我们采用同源权重继承策略。具体而言，我们使用主干网络的最后 L 层来初始化 Consolidator 的参数，其中 $L \ll N_{backbone}$。

这种初始化方式使 Consolidator 从一开始就处于与主干兼容的语义流形中，显著加速收敛，并促进更平滑的表示对齐。

随后，Consolidator 自回归地生成 K 个潜在嵌入 $\mathcal{M} = \{m_1, \ldots, m_K\}$，这些嵌入以软注入的方式融入主干的输入流中，以引导后续推理。

4 实验

4.1 实验设置

我们在六个基准上评估 FlashMem：推理（GSM8K (Cobbe et al., 2021)，MATH (Hendrycks et al., 2021)，GPQA (Rein et al., 2023)）、编码（KodCode (Xu et al., 2025b)）和摘要（BookSum (Kryscinski et al., 2022)，GovReport (Huang et al., 2021)）。我们与三种范式进行对比：（1）直接生成（Vanilla，CoT-SC (Wang et al., 2023)）；（2）KV 压缩（SnapKV (Li et al., 2024)）；以及（3）生成式潜在记忆（MemGen (Zhang et al., 2025)）。实验采用 Qwen（2.5-1.5B，3-4B）(Yang et al., 2024, 2025) 和 Llama（3.1-8B，3.2-3B）(Meta AI, 2024a,b) 的指令微调主干模型。针对每个主干模型，使用 SFT 训练一个轻量级 Consolidator，同时保持主干模型冻结；详细的实验协议和超参数见附录 A。

4.2 主要结果

表 1 和 2 分别展示了 FlashMem 在 Qwen 和 Llama 系列模型上的性能对比基准。

竞争性推理保真度。 FlashMem 在无需独立编码器的情况下，于推理任务中达到了与重型的 MemGen 基准相当的性能。例如，在 Qwen 2.5（表 1）上，FlashMem 在 GSM8K 上达到 **70.09%**，仅略低于 MemGen（70.54%），同时显著优于 Vanilla 基准。值得注意的是，在较小的 Llama 3.2 3B 模型上（表 2），FlashMem 在具有挑战性的 MATH 数据集上甚至超过了 MemGen（**48.05%** 对比 45.17%），这表明共

表 1: **Qwen 系列的结果**。我们在 Qwen 2.5（1.5B）和最新的 Qwen 3（4B）上将 FlashMem 与基准方法进行了比较。指标：推理/代码的准确率（%），摘要的 ROUGE-1。FlashMem 在保持更优效率的同时，始终达到或超过分离式潜在记忆基准（MemGen）的表现。最佳结果用 **粗体**标注，第二佳结果用 下划线 标注。

Dataset	Metric	Qwen 2.5 1.5B Instruct					Qwen 3 4B Instruct				
		Vanilla	CoT-SC	SnapKV	MemGen	FlashMem	Vanilla	CoT-SC	SnapKV	MemGen	FlashMem
Mathematical & Scientific Reasoning											
GSM8K	ACC	65.12	68.98	54.37	**70.54**	70.09	80.49	82.65	69.88	**85.42**	84.38
MATH	ACC	40.71	43.77	38.00	46.55	**50.16**	54.22	54.96	50.12	**60.23**	58.36
GPQA	ACC	13.96	15.14	13.66	**16.13**	14.73	16.84	17.95	14.54	**21.68**	20.54
Code Generation											
KodCode	ACC	22.85	29.30	39.70	**55.70**	50.20	56.48	57.55	49.70	59.77	**61.13**
Long-Context Summarization											
BookSum	R-1	11.14	11.62	10.38	12.86	**13.77**	8.93	9.35	**11.12**	10.18	10.99
GovReport	R-1	13.61	14.41	14.22	13.97	**16.11**	13.65	13.87	13.74	14.31	**15.52**

表 2: **Llama 系列结果**。 Llama 3.1（8B）和 Llama 3.2（3B）上的性能对比。FlashMem 在不同参数规模下表现出稳健的泛化能力，尤其在较小架构上的摘要生成和代码任务中表现突出。最佳结果以 **粗体**标出，次佳结果以 下划线 标出。

Dataset	Metric	Llama 3.1 8B Instruct					Llama 3.2 3B Instruct				
		Vanilla	CoT-SC	SnapKV	MemGen	FlashMem	Vanilla	CoT-SC	SnapKV	MemGen	FlashMem
Mathematical & Scientific Reasoning											
GSM8K	ACC	68.38	68.91	57.30	**71.12**	70.46	60.48	60.76	55.62	**63.68**	62.35
MATH	ACC	45.97	50.37	44.23	50.95	**51.56**	40.77	44.12	37.14	45.17	**48.05**
GPQA	ACC	15.48	17.80	15.66	**18.43**	17.86	4.79	5.18	5.27	**7.07**	6.70
Code Generation											
KodCode	ACC	34.61	37.90	40.88	**57.70**	54.27	21.93	26.27	20.33	29.64	**29.98**
Long-Context Summarization											
BookSum	R-1	13.12	14.62	11.67	**15.61**	14.65	9.03	9.29	10.25	**12.09**	11.04
GovReport	R-1	13.95	15.41	12.81	16.37	**17.23**	12.92	13.61	14.43	13.44	**14.55**

享-KV 设计可能在参数量较小的空间中更有效地保留数学先验。

长上下文优势与鲁棒性。 FlashMem 在摘要和结构密集型任务中表现出显著优势，在大多数骨干网络上持续优于 MemGen，特别是在 GovReport 和 KodCode 任务上。在 Qwen 3 4B 模型上，FlashMem 在 KodCode（**61.13%**）和 GovReport（**15.52%**）上均取得领先，表明直接从骨干网络的高保真 KV 缓存中提取记忆，比重新编码历史序列更能捕捉全局语义。此外，FlashMem 显著优于 KV 压缩基准 SnapKV（例如，在 Qwen 3 KodCode 上差距达 **+11%**），证实生成式潜在记忆对于保留复杂依赖关系至关重要，而启发式剪枝方法通常会丢弃这些信息。

4.3 效率分析

为实证验证 FlashMem 的架构效率，我们开展了一项压力测试，重点关注长上下文条件下计算延迟和内存开销。

基准协议。 我们在单块 NVIDIA A100-80GB GPU 上实现了一个 *循环生成基准*。评估采用 8 轮迭代模式，模型每轮生成 32 个文本 token，随后生成 8 个潜在记忆 token。我们将

表 3: **长上下文推理效率基准**。我们报告了 **峰值显存**使用量（最大观测值），以及 **有效生成吞吐量**（不包括潜在 token）和不同上下文长度（4k 到 64k）下的平均延迟。数值以均值 $\pm$Std. Dev. 表示。所有统计量均来自 30 次独立运行。背景颜色深浅表示相对于原始基线的性能开销；颜色越深，开销越高。FlashMem 的表现几乎与基线一致，而 MemGen 在长上下文下产生了难以接受的开销。

Context	Peak Memory (GB) ↓			Eff. Throughput (tokens/s) ↑			Latency (ms) ↓		
	Vanilla	MemGen	FlashMem	Vanilla	MemGen	FlashMem	Vanilla	MemGen	FlashMem
4k	4.61	13.19	7.29	$39.45_{\pm 3.84}$	$31.02_{\pm 2.87}$	$37.14_{\pm 0.61}$	$6.49_{\pm 0.94}$	$8.25_{\pm 1.02}$	$6.89_{\pm 0.10}$
8k	8.66	15.03	8.90	$38.29_{\pm 1.52}$	$26.34_{\pm 1.93}$	$37.28_{\pm 0.50}$	$6.68_{\pm 0.45}$	$9.72_{\pm 0.96}$	$6.87_{\pm 0.09}$
16k	11.88	18.71	12.12	$37.34_{\pm 2.57}$	$17.50_{\pm 1.16}$	$35.65_{\pm 2.78}$	$6.86_{\pm 0.82}$	$14.63_{\pm 0.99}$	$7.18_{\pm 0.68}$
32k	18.32	26.07	18.56	$34.24_{\pm 4.55}$	$9.98_{\pm 0.24}$	$29.74_{\pm 1.79}$	$7.48_{\pm 1.33}$	$25.65_{\pm 0.63}$	$8.61_{\pm 0.51}$
64k	31.21	40.78	31.44	$25.67_{\pm 1.79}$	$4.13_{\pm 0.06}$	$20.86_{\pm 1.01}$	$9.97_{\pm 0.83}$	$61.99_{\pm 0.88}$	$12.28_{\pm 0.48}$

在 Vanilla、MemGen (Zhang et al., 2025) 和 FlashMem 配置中，将输入上下文长度 L 从 4,096（4k）扩展到 65,536（64k）token。

效率结果。 如表 3 所示，MemGen 的分离式架构表现出较差的可扩展性：在 64k token 时，其显存占用上升至 **40.78 GB**（接近 10 GB 的开销），而由于重新编码瓶颈，有效吞吐量骤降至 **4.13 tokens/s**。相比之下，FlashMem 保持了与 Vanilla 基准几乎相同的高效性能（显存占用 31.44 GB vs 31.21 GB），同时维持了强劲的吞吐量（20.86 tokens/s）。关键的是，在最大上下文下的每步延迟方面，FlashMem（**12.28 ms**）相比 MemGen（61.99 ms）实现了约 **5**× 的加速，证实了共享-KV 设计有效消除了冗余存储和计算。

4.4 整合深度的影响

为了确定 Memory Consolidator 的最佳配置，我们研究了层深度 L（$1 \leq L \leq 6$）对推理性能和计算成本的影响。

表 4: **不同聚合器深度下的准确率**。我们在 GSM8K 和 KodCode 上报告了层深度从 $L = 1$ 到 6 时的准确率 (%)。性能保持稳定，表明增加深度带来的收益微乎其微。

Task	L=1	L=2	L=3	L=4	L=5	L=6
GSM8K	69.55	69.47	69.47	69.55	69.55	69.55
KodCode	50.10	50.19	50.29	50.19	50.29	50.29

性能饱和。 表 4 详细列出了在 GSM8K 和 KodCode 上的准确率。我们观察到性能迅速达到饱和：单层模型（$L = 1$）在 GSM8K 上达到了 69.55%，与更深的配置相当。类似地，在对结构敏感的 KodCode 任务中，将深度从 $L = 1$ 增加到 $L = 6$ 所带来的提升微乎其微（例如，50.10% 对比 50.29%）。这一实证证据强烈表明，单层 Consolidator 已足以从主干模型共享的 KV 缓存中提炼出高质量的潜在记忆，验证了我们极简设计的理念。

成本分析。 当准确率趋于平稳时，计算开销随深度增加而上升。如图 2 所示，显存(VRAM)使用量呈现严格的线性增长，与参数数量的增加直接相关。相比之下，推理延迟呈单调递增（吞吐量相应下降），反映了顺序解码累积的开销。鉴于 $L = 1$ 在资源开销最低的情况下提供了几乎相同的推理支持，我们得出结论：对于 FlashMem 框架而言，浅层架构是最优选择。

5 机制分析

5.1 认知监控的有效性

我们假设 *Cognitive Monitor* 充当不确定性检测器，识别出模型缺乏内部知识的关键决策点。为了验证这一点，我们分析了 GSM8K 数据集上的推理迹，特别关注了 Vanilla 模型失败而 FlashMem 成功的样本。

定性分析：熵动力学。 图 3 展示了四种不同主干网络的注意力熵轨迹。在这些情况下，原

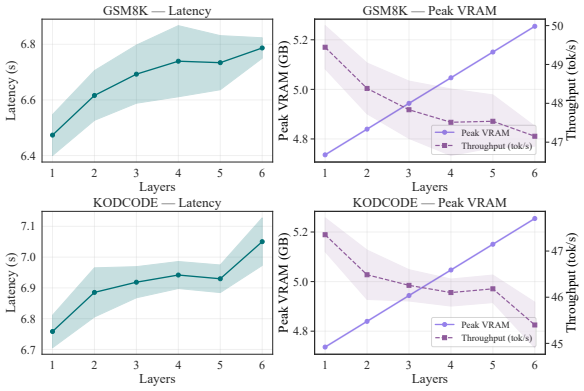

图 2: **不同层深度下的效率权衡。** VRAM 使用量随深度线性增长，而延迟则单调增加。结合表 4，结果倾向于采用最小深度配置。

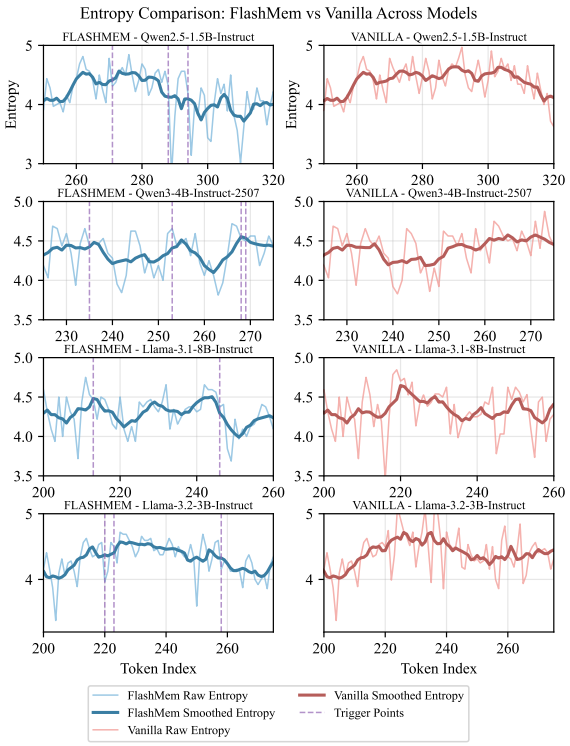

图 3: **推理过程中的熵动态。**我们在具有挑战性的 GSM8K 样本上，可视化了 FlashMem（蓝色）与 Vanilla（红色）在四个模型上的注意力熵。垂直线表示由认知监控器触发的记忆注入点。注入后，FlashMem 的熵明显低于基准，反映了更高的置信度和推理稳定性。

始基准模型在关键推理步骤中表现出持续或上升的熵值，表明存在认知混乱，最终导致逻辑错误。相比之下，FlashMem 能够成功识别这些高熵峰值。在注入合成的潜在记忆后，我们观察到注意力熵显著降低，反映出模型在后续 token 中内部不确定性的大幅减少。这证实了认知监控器能有效定位混淆时刻，而插入的记忆则有助于稳定推理路径。为严格验证这一对齐关系，我们进一步采用大语言模型作为评判员的方法来精确定位推理错误的起始时间，结果表明错误发生时间与检测到的熵峰值高度重合（见附录 B）。

定量分析：统计显著性。 为验证这种熵的降低在统计上是稳健的，我们计算了 GSM8K 测试集中所有触发实例的指标。

表 5: **通过记忆注入实现熵减少的统计量。**指标基于 GSM8K 中的触发步骤进行计算。高减少概率（86.5%）证实了认知监控在缓解模型不确定性方面的持续有效性。

Metric	Statistics
Mean Entropy Reduction	0.215 ± 0.125
Mean Relative Reduction	5.12% ± 3.85%
Entropy Reduction Range	-0.085 ∼ 0.850
Probability of Reduction	76.5%
Prob. of Significant Reduction (> 0.5)	7.2%

结果总结见表 5。平均而言，内存注入导致熵的均值降低 0.215，相对减少 5.12%。值得注意的是，熵降低的概率高达 76.5%，表明在绝大多数情况下，访问潜在内存能够有效解决模型内部的冲突。此外，在 7.2% 的情况下出现显著的熵下降（降低 > 0.5），这些情况通常对应着洞察的关键时刻，此时综合记忆纠正了潜在的幻觉。

5.2 可视化记忆巩固过程

为了进一步探究 FlashMem 如何弥合远距离上下文与当前生成之间的差距，我们可视化了 Consolidator 与主干网络的内部注意力动态。由于篇幅限制，详细的定性分析及相应的注意力热图见附录 D。其中的观察结果证实，Consolidator 起到了有效的语义过滤作用，而主干网络在整个生成过程中对注入的潜在 token 保持了持续的关注。

6 结论

在本文中，我们提出了 **FlashMem**，一种协调自主智能体中高密度潜在记忆与推理效率之间矛盾的框架。通过用 **共享-键值整合器** 替代独立的辅助编码器，FlashMem 通过对主干网络冻结表示进行计算复用，直接提炼记忆，消除了冗余重参数化的开销。此外，集成一个无参数的 **认知监控器**使智能体能够根据实时不确定性动态触发记忆整合，确保资源分配的合理性。大量实验表明，FlashMem 在保持与重型基准相当的推理保真度的同时，将推理延迟降低了高达 5× 。我们希望这项工作能建立一种新的内在高效记忆机制范式，推动长时程智能体在资源受限环境中的部署。

局限性

尽管 FlashMem 展现出强大的效率和推理性能提升，但仍存在未来研究的拓展空间。首先，当前框架主要针对文本和代码类任务进行了优化，尚未将共享键值（Shared-KV）范式扩展至多模态情景。鉴于视觉-语言模型（VLMs）的重要性日益凸显，将 FlashMem 适配以处理多感官输入及视觉 token，是未来工作的一个有前景方向。其次，虽然我们已在代表性开源模型上验证了 FlashMem 的有效性，但其内在记忆机制在超大规模基础模型（如参数量达数百亿级别的模型）上的探索仍待开展。在如此规模下研究计算复用的缩放特性，将有助于全面揭示潜在记忆的潜力。

道德考虑

本工作遵循绿色人工智能原则，通过高效的计算复用和降低推理延迟显著减少能耗。我们严格遵守实验中所用开源模型和数据集的许可协议与使用政策。尽管继承了预训练主干网络的固有特性，但我们的框架不会引入数据隐私或侵犯人权方面的额外风险。

References

Adrien Bibal, Rémi Cardon, David Alfter, Rodrigo Wilkens, Xin Wang, Patrick Watrin, and Thomas Schneider. 2022. Attention entropy of large language models. In *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (ACL)*, pages 201–212.

Sebastian Borgeaud, Arthur Mensch, Jordan Hoffmann, Trevor Cai, Eliza Rutherford, Katie Millican, and 1 others. 2022. Improving language models by retrieving from trillions of tokens. In *International Conference on Machine Learning (ICML)*, pages 2206–2240.

Zefan Cai, Yichi Zhang, Bofei Gao, Yuliang Liu, Yucheng Li, Tianyu Liu, Keming Lu, Wayne Xiong, Yue Dong, Junjie Hu, and Wen Xiao. 2025. Pyramidkv: Dynamic kv cache compression based on pyramidal information funneling. *arXiv preprint arXiv:2406.02069*.

Prateek Chhikara, Dev Khant, Saket Aryan, Taranjeet Singh, and Deshraj Yadav. 2025. Mem0: Building production-ready ai agents with scalable long-term memory. *arXiv preprint arXiv:2504.19413*.

Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, and 1 others. 2021. Training verifiers to solve math word problems. *arXiv preprint arXiv:2110.14168*.

Zihang Dai, Zhilin Yang, Yiming Yang, Jaime Carbonell, Quoc Le, and Ruslan Salakhutdinov. 2019. Transformer-xl: Attentive language models beyond a fixed-length context. In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics (ACL)*, pages 2978–2988.

Timothée Darcet, Maxime Oquab, Julien Mairal, and Piotr Bojanowski. 2024. Vision transformers need registers. In *International Conference on Learning Representations (ICLR)*.

Sebastian Farquhar, Jannik Kossen, Lorenz Kuhn, and Yarin Gal. 2024. Detecting hallucinations in large language models using semantic entropy. *Nature*, 630(8017):625–630.

Suyu Ge, Yunan Zhang, Liyuan Liu, Minjia Zhang, Jiawei Han, and Jianfeng Gao. 2024. Model tells you what to discard: Adaptive KV cache compression for LLMs. In *International Conference on Learning Representations (ICLR)*.

Mor Geva, Roei Schuster, Jonathan Berant, and Omer Levy. 2021. Transformer feed-forward layers are key-value memories. In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 5484–5495.

Nuno M Guerreiro, Elena Voita, and André FT Martins. 2023. Looking for a needle in a haystack: A comprehensive study of hallucinations in neural machine translation. In *Proceedings of the 17th Conference of the European Chapter of the Association for Computational Linguistics (EACL)*, pages 1059–1075.

Dan Hendrycks, Collin Burns, Saurav Kadavath, Akul Arora, Steven Basart, Eric Tang, and Dawn Song. 2021. Measuring mathematical problem solving with the math dataset. *Advances in Neural Information Processing Systems*, 34:25129–25141.

Yuyang Hu, Shichun Liu, Yanwei Yue, Guibin Zhang, Boyang Liu, Fangyi Zhu, Jiahang Lin, Honglin Guo, Shihan Dou, Zhiheng Xi, and 1 others. 2025. Memory in the age of ai agents: A survey. *arXiv preprint arXiv:2512.13564*.

Luyang Huang, Shuyang Cao, Nikolaus Parulian, Heng Ji, and Lu Wang. 2021. Efficient attentions for long document summarization. In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*, pages 7779–7793.

Saurav Kadavath, Tom Conerly, Amanda Askell, Tom Henighan, Dawn Drain, Ethan Perez, Nicholas Schiefer, Zac Hatfield-Dodds, and 1 others. 2022. Language models (mostly) know what they know. *arXiv preprint arXiv:2207.05221*.

Takeshi Kojima, Shixiang Shane Gu, Machel Reidsma, and Tatsuya Harada. 2022. Large language models are zero-shot reasoners. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 35, pages 22199–22213.

Wojciech Kryscinski, Nazneen Rajani, Divyansh Agarwal, and Caiming Xiong. 2022. Booksum: A collection of datasets for long-form narrative summarization. In *Findings of the Association for Computational Linguistics: EMNLP 2022*, pages 2988–3002.

Lorenz Kuhn, Yarin Gal, and Sebastian Farquhar. 2023. Semantic uncertainty: Linguistic invariances for uncertainty estimation in natural language generation. In *International Conference on Learning Representations (ICLR)*.

Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, and 1 others. 2020. Retrieval-augmented generation for knowledge-intensive nlp tasks. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 33, pages 9459–9474.

Hengli Li, Chenxi Li, Tong Wu, Xuekai Zhu, Yuxuan Wang, Zhaoxin Yu, Eric Hanchen Jiang, Song-Chun Zhu, Zixia Jia, Ying Nian Wu, and Zilong Zheng. 2025. Seek in the dark: Reasoning via test-time instance-level policy gradient in latent space. *arXiv preprint arXiv:2505.13308*.

Yuhong Li, Yingbing Huang, Bowen Yang, Bharat Venkitesh, Acyr Locatelli, Hanchen Ye, Tianle Cai, Patrick Lewis, and Deming Chen. 2024. Snapkv: Llm knows what you are looking for before generation. In *Advances in Neural Information Processing Systems*, volume 37.

Stephanie Lin, Jacob Hilton, and Owain Evans. 2022. Teaching models to express their uncertainty in words. *Transactions on Machine Learning Research*, pages 96–107.

Zichang Liu, Aditya Desai, Fangshuo Xu, Ryan Wang, Yuan Xie, and Zhiru Xu. 2023. Scissorhands: Exploiting the persistence of importance hypothesis for LLM KV cache compression at inference. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 36.

Kevin Meng, David Bau, Alex Andonian, and Yonatan Belinkov. 2022. Locating and editing factual associations in GPT. In *Advances in Neural Information Processing Systems*, volume 35, pages 17359–17372.

Meta AI. 2024a. Llama 3.1 technical report. *Meta AI Technical Report*.

Meta AI. 2024b. Llama 3.2 technical report. *Meta AI Technical Report*.

Jesse Mu, Xiang Lisa Li, and Noah D Goodman. 2023. Learning to compress prompts with gist tokens. In *Advances in Neural Information Processing Systems*, volume 36, pages 12653–12671.

Giorgos Nikolaou, Tommaso Mencattini, Donato Crisostomi, Andrea Santilli, Yannis Panagakis, and Emanuele Rodolà. 2025. Language models are injective and hence invertible. *arXiv preprint arXiv:2510.15511*.

Charles Packer, Vivian Fang, Shishir G Patil, Kevin Lin, Sarah Wooders, and Joseph E Gonzalez. 2023. MemGPT: Towards LLMs as operating systems. *arXiv preprint arXiv:2310.08560*.

David Rein, Betty Li Hou, Asa Hutchinson, Armen Aghajanyan, Sam Clark, Hannaneh Hajishirzi, Luke Zettlemoyer, and Fabio Petroni. 2023. Gpqa: A graduate-level google-proof q&a benchmark. *arXiv preprint arXiv:2311.12022*.

Ian Tenney, Dipanjan Das, and Ellie Pavlick. 2019. Bert rediscovers the classical nlp pipeline. In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*, pages 4593–4601.

Xuezhi Wang, Jason Wei, Dale Schuurmans, Quoc Le, Ed Chi, Sharan Narang, Aakanksha Chowdhery, and Denny Zhou. 2023. Self-consistency improves chain of thought reasoning in language models. In *International Conference on Learning Representations (ICLR)*.

Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed Chi, Quoc Le, and Denny Zhou. 2022. Chain-of-thought prompting elicits reasoning in large language models. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 35, pages 24824–24837.

Wenyi Wu, Zixuan Song, Kun Zhou, Yifei Shao, Zhiting Hu, and Biwei Huang. 2025. Towards general continuous memory for vision-language models. *arXiv preprint arXiv:2505.17670*.

Yuhuai Wu, Markus N Rabe, DeLesley Hutchins, and Christian Szegedy. 2022. Memorizing transformers. In *International Conference on Learning Representations (ICLR)*.

Guangxuan Xiao, Yuandong Tian, Beidi Chen, Song Han, and Mike Lewis. 2024. Efficient streaming language models with attention sinks. In *International Conference on Learning Representations (ICLR)*.

Yige Xu, Xu Guo, Zhiwei Zeng, and Chunyan Miao. 2025a. Softcot: Soft chain-of-thought for efficient reasoning with llms. *arXiv preprint arXiv:2502.12134*.

Zhihong Xu, Yang Liu, Chen Li, Xipeng Qiu, and Xuanjing Huang. 2025b. Kodcode: A diverse, challenging, and verifiable synthetic dataset. *arXiv preprint arXiv:2502.16748*.

A Yang and 1 others. 2024. Qwen2.5 technical report. *arXiv preprint arXiv:2412.15115*.

A Yang and 1 others. 2025. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*.

Xinlei Yu, Chengming Xu, Guibin Zhang, Zhangquan Chen, Yudong Zhang, Yongbo He, Peng-Tao Jiang, Jiangning Zhang, Xiaobin Hu, and Shuicheng Yan. 2025. Vismem: Latent vision memory unlocks potential of vision-language models. *arXiv preprint arXiv:2511.11007*.

Guibin Zhang, Muxin Fu, and Shuicheng Yan. 2025. Memgen: Weaving generative latent memory for self-evolving agents. *arXiv preprint arXiv:2509.24704*.

Zhenyu Zhang, Ying Sheng, Tianyi Zhou, Tianlong Chen, Lianmin Zheng, Ruisi Cai, Zhao Song, Yuandong Tian, Christopher Ré, Clark Barrett, Zhangyang Wang, and Beidi Chen. 2023. H2O: Heavy-hitter oracle for efficient generative inference of large language models. In *Advances in Neural Information Processing Systems*, volume 36, pages 34661–34701.

Guorui Zhou, Hengrui Hu, Hongtao Cheng, Huanjie Wang, Jiaxin Deng, Jinghao Zhang, Kuo Cai, Lejian Ren, and 1 others. 2025a. OneRec-V2 technical report. *arXiv preprint arXiv:2508.20900*.

Huichi Zhou, Yihang Chen, Siyuan Guo, Xue Yan, Kin Hei Lee, Zihan Wang, Ka Yiu Lee, Guchun Zhang, Kun Shao, Linyi Yang, and Jun Wang. 2025b. Memento: Fine-tuning LLM agents without fine-tuning LLMs. *arXiv preprint arXiv:2508.16153*.

A 实验实施细节

A.1 数据构建与训练目标

领域自适应专家蒸馏 为确保记忆巩固器能够捕捉高保真度的领域特定推理模式，我们超越了通用预训练语料库。我们采用了一种目标导向的专家蒸馏策略。针对每个下游领域——具体为数学推理（GSM8K, MATH）和代码生成（KodCode）——我们精心构建了一个高质量执行轨迹数据集。为实现这一目标，我们利用先进的专有模型合成思维链（CoT）推理过程。这些合成的轨迹经过严格的拒绝采样过滤，仅保留那些通过符号执行验证代码正确性或通过确定性等价性检查验证数学正确性的轨迹。该过滤过程确保巩固器仅在正向强化信号上进行训练，最大限度降低学习到幻觉或次优推理路径的风险。

序列构建与掩码。 在最优化阶段，我们为每个实例构建一个统一的训练序列 S。该序列连结了初始指令提示 x、合成的潜在记忆占位符 $\mathcal{M}_{gen}$ 以及真实专家轨迹 y：

$$S = [x, \mathcal{M}_{gen}, y] \tag{6}$$

此处，$\mathcal{M}_{gen}$ 表示由整合器生成的 K 个潜在向量序列。关键的是，我们在损失计算中应用了一种策略性掩码协议。对应于指令 token x 和潜在记忆 token $\mathcal{M}_{gen}$ 的标签索引被设置为 -100，这在 PyTorch 中作为标准忽略索引。因此，损失仅在专家轨迹 y 的预测上进行计算。

梯度流与最优化 训练目标是使在注入记忆条件下的目标 token y 上的标准交叉熵损失（$\mathcal{L}_{CE}$）最小化：

$$\mathcal{L} = -\sum_{t=1}^{|y|} \log P_\theta(y_t \mid x, \mathcal{M}_{gen}, y_{<t}) \tag{7}$$

我们实现中的一个关键方面是计算图。主干网络参数 θ 完全冻结（$\nabla_\theta = 0$）。梯度从损失函数反向传播，经过主干网络的冻结层，仅流入软记忆嵌入 $\mathcal{M}_{gen}$。随后，这些梯度流入整合器的参数 ψ。这种设置迫使整合器学习一种隐式目标：它必须将信息编码到 $\mathcal{M}_{gen}$，使得冻结的主干网络生成正确专家轨迹 y 的似然性最大化。这实际上起到了来自未来的知识蒸馏作用，将轨迹 y 的语义精髓压缩至潜在状态 $\mathcal{M}_{gen}$。

A.2 超参数和硬件环境

基础设施与软件栈。 所有训练和推理实验均在配备 4 × NVIDIA A100-SXM4-80GB GPU 的高性能计算结点上进行。该框架使用 PyTorch 2.7.1 和 Hugging Face Transformers 库实现。为了优化训练吞吐量并管理较大主干模型的内存占用，我们采用了 DeepSpeed ZeRO-2 最优化和 FlashAttention-2 内核。这使得我们能够在小规模实验中无需对模型进行完全分片即可高效训练 Consolidator。

配置合理性说明。 表 6 提供了超参数的全面列表。我们选择内存 token 数量为 $K = 8$ 作为我们架构的最优工作点。关于 Consolidator 的深度，基于第 4.4 节中详述的效率基准，我们将 $L = 1$ 设定为优先考虑最小的显存开销和最大的吞吐量。为缓解梯度爆炸——一种自回归内存生成中的已知不稳定性——我们施加了严格的梯度裁剪范数 0.53。学习率调度采用前 10% 步数的线性预热，随后采用余弦衰减，以确保从主干网络继承的参数在预训练语义空间内稳定适应。

A.3 推理动态与算法

FlashMem 的运行时行为由一个交互环控制，该环在标准 token 生成和潜在记忆整合之间交替进行。我们在算法 1中提供了形式化的伪代码。

确定性内存生成。 与用于文本生成的标准随机采样不同，记忆生成步骤（算法 1 中的第 8 行）采用严格的贪心搜索策略。这种确定性方法具有两个目的：首先，它稳定了潜在空间，防止将高方差噪声注入主干模型的上下文；其次，

表 6: FlashMem 训练与推理的超参数。

Hyperparameter	Value
Model Architecture	
Memory Token Count (K)	8
Consolidator Layers (L)	1
Hidden Dimension	Matches Backbone
Training Optimization	
Optimizer	AdamW
Learning Rate	1e-5
Weight Decay	0.01
Gradient Clipping	0.53
Global Batch Size	64
Training Epochs	5
Scheduler	Cosine Decay
Warmup Ratio	0.1
Inference	
Memory Decoding Strategy	Greedy Search (Deterministic)
Entropy Threshold (τ)	Adaptive (85th percentile)

在给定相同上下文状态的情况下，确保推理轨迹的可复现性。

缓存管理与注入 一个关键的实现细节在于对键值（KV）缓存（第 10 行）的管理。当内存 $\mathcal{M}$ 被合成时，其并非仅仅与输入提示字符串连结。相反，$\mathcal{M}$ 的连续嵌入会通过主干网络生成对应的 KV 对，然后被追加到活跃的 KV 缓存 $\mathcal{K}, \mathcal{V}$ 中。这使得主干网络在后续所有步骤中都能像处理物理上下文窗口中的内容一样，关注这些虚拟记忆 token，而无需重新计算先前的历史。

Algorithm 1 FlashMem Inference Loop

1: **Input:** Prompt x, Backbone π_θ, Consolidator $\mathcal{C}_\psi$, Threshold τ
2: **Initialize:** KV Cache $\mathcal{K}, \mathcal{V} \leftarrow \emptyset$, Context $c \leftarrow x$
3: **while** not end of generation **do**
4: Logits, Attentions, $h_{last} \leftarrow \pi_\theta(c, \mathcal{K}, \mathcal{V})$
5: $\mathcal{H}_{current} \leftarrow$ CalcEntropy(Attentions)
6: **if** $\mathcal{H}_{current} > \tau$ **then** ▷ High Uncertainty Detected
7: $m_0 \leftarrow$ Project(h_{last})
8: $\mathcal{M} \leftarrow$ GreedyGen($\mathcal{C}_\psi, m_0, \mathcal{K}, \mathcal{V}$, steps $= 8$)
9: **Inject:** Soft-append $\mathcal{M}$ to Backbone Input
10: Update $\mathcal{K}, \mathcal{V}$ with $\mathcal{M}$ states
11: **end if**
12: $x_{next} \leftarrow$ Sample(Logits)
13: $c \leftarrow x_{next}$
14: **end while**

B 认知监测分析方法

本节详细描述了用于验证认知监测器有效性的实验方案。这些方案支撑了第 5.1 节中提出的定量与定性分析，特别关注注意力熵与推理保真度之间的相关系数。

B.1 通过大模型作为裁判进行错误定位

为了验证检测到的高熵峰值是否对应真实的认知断点和内部模型混淆，我们在 GSM8K 数据集上进行了细粒度的错误分析。

自动化评估设置。 在数千条轨迹上手动标注推理错误的精确发生时间成本过高，且容易受到主观偏见的影响。因此，我们采用 Gemini 3 Pro 作为客观的自动化评估者（大模型作为裁判）。我们指示该裁判审计未经增强的原始模型生成的推理轨迹，其目标是准确找出逻辑首次偏离真实值或出现内部不一致的具体推理步骤。

本次评估所使用的提示旨在强制执行严格的逐步验证，如表 7 所示。

时间对齐分析 我们收集了裁判返回的错误时间戳，并将其与生成原始轨迹过程中记录的时间序列熵日志对齐。这种对齐揭示了一种显著的同步模式：注意力熵的大幅波动始终在被标记为错误的步骤之前或同时出现。这一实证证据验证了我们的核心假设，即注意力熵可作为模型困惑度的可靠实时代理，为原始基准模型中观察到的不稳定高熵行为提供了物理基础（图 3）。

B.2 熵减的统计协议

表 5 在正文报告了 FlashMem 对降低模型不确定性的影响。此处，我们定义了一组配对控制实验，旨在隔离记忆注入对注意力熵的因果效应。

数据构建。 为确保公平比较，我们采用双运行数据采集策略。我们在同一组评估提示上，从两种不同的执行模式中收集了推理迹：

System Prompt: You are an expert Mathematics Evaluator. Your task is to inspect a given solution trajectory for a math problem and identify the first step where a logical or calculation error occurs.

Input:

- **Question:** [Input Question]
- **Gold Solution:** [Ground Truth Reference]
- **Model Trajectory:** [Model's Reasoning Output]

Instructions:

1. Compare the Model Trajectory against the Gold Solution step-by-step.
2. Identify the first step index (0-indexed) where the reasoning is flawed. If the reasoning is strictly correct but diverges in method, do not count it as an error unless it leads to a wrong answer.
3. If the entire trajectory is correct, return -1.

Output Format:

Return a JSON object: {"error_found": boolean, "first_error_step": int, "reason": "string"}

表 7: 用于大语言模型作为裁判进行错误定位分析的评估提示。

- FlashMem 轨迹（处理组）：从配备完整 FlashMem 机制的模型中收集的数据。这些迹包括一个增强掩码，该掩码显式记录由认知监控触发的每次内存注入事件的时间戳 t。
- 原始轨迹（对照组）：从基础模型在未进行记忆增强的情况下收集的数据。我们强制模型处理相同的提示，以原始生成结果作为无干预情况下熵水平的反事实基准。

基于窗口的比较分析。 熵是一个随 token 独特性自然波动的动态指标。为系统量化记忆注入的即时局部影响并剔除全局噪声，我们采用了一种基于窗口的双重差分统计方法：

1. 触发识别：对于 FlashMem 组中的每个样本，我们识别了与认知监控器确定 $\mathcal{H} > \tau$ 时增强掩码激活对应的触发索引 t。有效触发被限制在索引 $t > 5$ 之间，以减轻初始生成不稳定性带来的混杂影响。
2. 窗口定义：我们建立了一个局部分析窗口 W，其固定的时间范围为触发点之后的 $L = 10$ 个 token，涵盖了区间 $[t, t + L]$。该窗口捕捉了注入记忆对主干网络内部状态的即时下游影响。
3. 指标计算：我们计算了该特定窗口内 FlashMem 轨迹的注意力熵均值 $\mathcal{H}$，并将其与在相同位置索引下测量的 Vanilla 基准的注意力熵均值进行对比。这种逐点比较消除了由位置相关熵变化引起的方差影响。

量化指标。 每个触发样本 i 的净减少效应通过反事实基准与处理状态之间的差异来量化：

$$\Delta\mathcal{H}_i = \mathcal{H}^{(i)}_{\text{vanilla}}[t : t + L] - \mathcal{H}^{(i)}_{\text{flashmem}}[t : t + L] \tag{8}$$

一个正的 $\Delta\mathcal{H}_i$ 表示内存注入成功降低了模型不确定性。基于 $\Delta\mathcal{H}_i$ 的分布，我们得到了以下统计指标：

- 减少概率：定义为 $P(\Delta\mathcal{H}_i > 0)$。这表示记忆注入成功稳定注意力分布的频率。
- 显著减少的概率：在 $\Delta\mathcal{H}_i > \tau_{sig}$（具有 $\tau_{sig} = 0.5$）的样本中所占的比例。这些情况通常对应于合成记忆解决主要分歧的关键时刻。
- 相对减少百分比：一种规范化的度量，计算公式为 $\frac{\Delta\mathcal{H}_i}{\mathcal{H}^{(i)}_{\text{vanilla}}} \times 100\%$，反映了改进程度相对于初始混淆水平的大小。

这种配对设计确保了观察到的熵降低与 FlashMem 干预存在因果关系，证实了潜在上下文直接提升了模型的确定性，而非令牌位置的产物。

C 熵动力学分析与阈值选择

认知监控器通过决策阈值 τ 调节记忆巩固的启动。我们的实证分析表明，绝对注意力

熵并非一个普适常数；相反，它会显著地随潜在模型架构和提示的结构特性而波动。在本节中，我们阐述了这些主导动态，并详细说明用于自适应阈值确定的统计方法。

C.1 注意力熵的实证趋势

模型尺度与置信度 我们观察到模型参数量与内在熵水平之间存在明显的负相关。较大的语言模型通常表现出更尖锐的注意力分布，将概率质量集中于更少的 token 上。这种行为反映了在下一个 token 预测上相较于小型架构具有更高的内部置信度，而小型架构则往往表现出更分散的注意力模式。因此，适用于小型模型的静态阈值对于大型模型而言灵敏度不足。为了保持一致的监控性能，τ 的绝对值必须与模型大小成反比变化，从而确保系统对置信度相对下降的响应能力，而非绝对数值的变化。

任务拓扑与解空间 基准熵谱进一步受到任务解空间拓扑结构的调制。具有广阔、开放性搜索空间的任务，如代码生成或创意写作，由于存在多种发散的生成路径且均有效，自然会引起更高的熵波动。相比之下，需要收敛推理的任务，如算术或严格逻辑，则通常处于较低熵的运行状态。摘要生成和通用知识检索一般介于这两者之间。这些差异表明，高不确定性状态的定义是相对的：在数学任务中表示困惑的特定熵值，在创意写作情境中可能仅代表正常运行状态。

C.2 分布感知的选择策略

鉴于这些架构和任务相关的差异，依赖固定的启发式 τ 值是次优的。我们改而采用一种分布感知策略，以动态校准活性值边界。

我们将记忆注入概念化为一种专门针对认知异常值的干预机制——即模型在置信度上显著偏离其标准基准的情况。为了准确捕捉这些异常值，我们在推理之前基于代表性验证子集上观察到的熵分布对 τ 进行校准。形式上，阈值通过百分位函数确定：

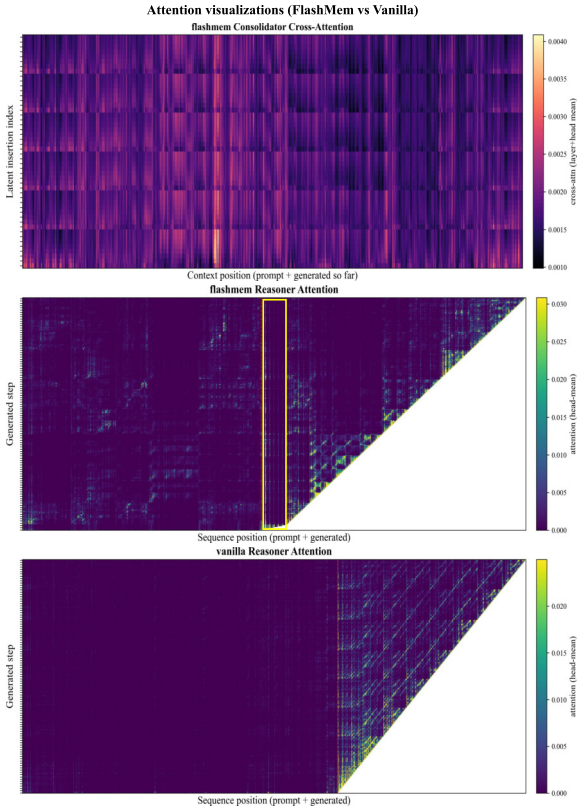

图 4: **注意力动态可视化。 顶部：**整合器的交叉注意力能够准确地聚焦于上下文中的关键信息（例如，变量定义）。**中间：** FlashMem 的主干网络在整个生成过程中持续关注注入的潜在 token（用黄色方框标记），这由连续的垂直注意力线所证实。**底部：**传统模型表现出分散的注意力，无法捕捉远距离依赖关系。

$$\tau = \text{Percentile}(\{\mathcal{H}_i\}_{i=1}^{N}, P_{\text{target}}) \qquad (9)$$

在我们的实验中，我们配置 P_{target} 以目标分布的上尾部分，具体为第 85 百分位。此配置作为一种有效的过滤器，能够抑制常规生成噪声，同时保留对显著不确定性峰值的敏感性。这些峰值通常对应于模型需要外部记忆支持的关键决策点。这种统计方法使 FlashMem 能够自动根据模型和任务分布的特定特性调整绝对阈值，从而无需手动微调。

D 注意力动态的详细可视化

在本节中，我们提供了关于第 5 节中提到的 KodCode 错误案例的详细定性分析和可视

化。

通过交叉注意力进行语义提取。 图 4 的上半部分展示了在潜在 token 生成过程中记忆整合器的交叉注意力权重。我们观察到，整合器表现出高度选择性注意力，强烈聚焦于输入上下文的特定区域（热力图中已高亮显示）。详细检查发现，这些高权重区域与长历史中的语义显著片段相吻合，这些片段对当前预测至关重要。这证实了共享键值（Shared-KV）机制有效充当了语义过滤器，将原始历史中的关键依赖关系提取并压缩至潜在空间。

通过自注意力实现持续参照。 中间面板可视化了 FlashMem 主干在后续文本生成阶段的自注意力机制。为便于观察，我们在固定间隔处插入了潜在 token（用黄色方框标记）。红色虚线划分了输入提示与模型生成文本之间的边界。一个显著的模式浮现：有五个清晰的垂直注意力带，与插入的潜在 token 完全对齐。这表明在整个生成过程中，主干网络始终会回关注这些潜在嵌入。这些记忆 token 有效地充当高密度语义锚点，为持续推理提供持续支持。

与基础基准的比较。 相比之下，缺乏潜在记忆引导时，注意力分布明显更加分散，缺乏结构上的聚焦。模型无法持续关注远处的变量定义（地图中显示为深色区域），导致关键上下文丢失，进而产生未定义变量的幻觉。这一对比有力验证了我们的假设：FlashMem 的成功并非仅仅通过扩展上下文长度实现，而是通过构建显式的、可访问的语义桥梁，使模型能够主动利用这些信息。

E 扩展定性分析

在本节中，我们利用 GSM8K 基准对数学推理中的失败模式进行了详细分析。我们从测试集中选取了三个具有代表性的案例，其中基准模型因输入幻觉、变量绑定错误或过度推理而出现失败。

在以下比较表格中，我们明确标识了 FlashMem 认知监控触发潜在记忆注入的决策点。这些干预措施有助于刷新关键上下文，并将推理轨迹引导回正确的逻辑路径，相较于基准情况。

E.1 输入信息幻觉的纠正

推理任务通常需要从自然语言描述中精确提取数值。一种常见的失败模式是输入幻觉，即模型正确识别了实体，但根据内部先验赋予了错误的数值。

表 8（案例 1）说明了一个时间计算问题。提示中指出，削土豆需要“一分半钟”(90 秒)。基准模型却幻觉出“1 分钟 15 秒”的数值。在 FlashMem 轨迹中，记忆机制在单位转换步骤立即触发。这种注入起到了语义锚点的作用，强制模型遵循输入历史中的特定文本片段，从而覆盖了幻觉结果。

E.2 变量绑定的修正

复杂问题通常涉及人群的多个子集。模型在变量绑定方面经常遇到困难，即在中间计算之后，将某个比率或率应用到了错误的子集上。

表 8（情形 2）展示了一个派对策划场景。约束条件是明确的：每位“受邀宾客”都需要准备礼品袋。基准模型虽然正确计算了参会人数，但随后错误地将礼品袋比例应用到了这一新数量上。对于 FlashMem，记忆注入发生在参会人数计算之后，这一干预重新激活了关于受邀宾客的原始约束，从而纠正了最终成本估算中操作数的选择。

E.3 语义过度推理的缓解

大型语言模型有时会过度推理，将简单的问题陈述误解为陷阱题，或应用与数学抽象相矛盾的无关现实约束。

表 8（案例 3）详细描述了一个关于鸡蛋消耗量的问题。基准模型正确执行了算术运算，但在最后一步拒绝了结果，理由是“制作”不是“食用”。FlashMem 执行了相同的计算，但在接近结论时，记忆注入使模型的注意力集中在量化累积任务上，抑制了语义拒绝的生成。

Case 1: Input Information Hallucination

Problem: Billy has 60 potatoes. It takes him **a minute and a half** to peel a potato, and 5 seconds to cut it. How long will it take to finish prepping the potatoes?

Baseline Generation	FlashMem Generation
Let's break down the time per potato.	*[Latent Memory Injection Triggered]*
First, convert the peeling time.	↪ *Retrieving precise duration: "minute and a half" implies 90s.*
1 minute and 15 seconds = 75 seconds.	Let's break down the time per potato.
The time to cut is 5 seconds.	First, convert the peeling time.
	1 minute and 30 seconds = 60 + 30 = 90 seconds.
So, total time per potato:	The time to cut is 5 seconds.
$75 + 5 = 80$ seconds.	So, total time per potato:
	$90 + 5 = 95$ seconds.
Now for 60 potatoes:	Now for 60 potatoes:
$60 \times 80 = 4800$ seconds.	$60 \times 95 = 5700$ seconds.
Convert to minutes: $4800/60 = 80$.	Convert to minutes: $5700/60 = 95$.
Final Answer: 80	Final Answer: 95

Case 2: Variable Binding Error

Problem: Christina needs 0.75 gift bags per **invited guest**. She invited 16 friends, but 1/4 don't show up. Gift bags are $2 each. How much will she spend?

Baseline Generation	FlashMem Generation
Step 1: Determine guests attending.	Step 1: Determine guests attending.
Total invited = 16.	Total invited = 16.
Guests not showing up $= 1/4 \times 16 = 4$.	Guests not showing up $= 1/4 \times 16 = 4$.
Guests attending $= 16 - 4 = 12$.	Guests attending $= 16 - 4 = 12$.
Step 2: Determine gift bags needed.	*[Latent Memory Injection Triggered]*
We need 0.75 bags per guest.	↪ *Refreshing constraint: Ratio applies to "Invited" (16), not "Attending".*
So, $0.75 \times 12 = 9$ bags.	Step 2: Determine gift bags needed.
	The problem states 0.75 bags per **invited guest**.
	So, $0.75 \times 16 = 12$ bags.
Step 3: Calculate total cost.	Step 3: Calculate total cost.
$9 \times \$2 = \$18$.	$12 \times \$2 = \$24$.
Final Answer: 18	Final Answer: 24

Case 3: Semantic Over-Reasoning

Problem: Claire makes a 3 egg omelet every morning. How many dozens of eggs will she eat in 4 weeks?

Baseline Generation	FlashMem Generation
Step 1: Calculate daily usage.	Step 1: Calculate daily usage.
She uses 3 eggs per day.	She uses 3 eggs per day.
Step 2: Calculate 4 weeks usage.	Step 2: Calculate 4 weeks usage.
4 weeks = 28 days.	4 weeks = 28 days.
Total eggs $= 3 \times 28 = 84$ eggs.	Total eggs $= 3 \times 28 = 84$ eggs.
Step 3: Convert to dozens.	Step 3: Convert to dozens.
$84/12 = 7$ dozens.	$84/12 = 7$ dozens.
Step 4: Final Verification.	*[Latent Memory Injection Triggered]*
Wait, the question asks what she will *eat*.	↪ *Suppressing distractor: "Making" implies "Eating" in this math context.*